

```
In [ ]: import marimo as mo
import pandas as pd
from ortools.linear_solver import pywraplp
```

Trabalho Prático: Horário Escolar

A Biblioteca de modelação escolhida foi OR-Tools, uma vez que foi usada para resolver um problema do mesmo tipo numa aula prática.

Utilizei a biblioteca "pandas" para ler os dados em formato csv uma vez que tem algumas funcionalidades úteis, como por exemplo, o `fillna()` para substituir os valores `NaN` por outro.

Reutilizei código que foi feito numa das aulas práticas para representar informação de um problema parecido. Foram feitas umas pequenas alterações para mostrarmos os horários.

Leitura de dados (R8)

Os dados são lidos com `pandas`, como foi referido anteriormente, a partir dos ficheiros `csv` da pasta indicada. As únicas constantes são `dias` e `periodos` e todos os outros dados são lidos dos ficheiros.

```
In [ ]: periodos = 5
dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
```

```
In [ ]: def read_data(filename, replace_values=[]):
    data = pd.read_csv(filename)
    for col, value in replace_values:
        data[col] = data[col].fillna(value)
    return data.values

salas = {}
sala_default = ""
for s in read_data("dados/salas.csv"):
    salas[s[0]] = s[2]
    if s[1] == "normal":
        sala_default = s[0]

turmas = [t[0] for t in read_data("dados/turmas.csv")]

disciplinas = {}
professores = {}
for _d in read_data("dados/disciplinas.csv", [{"sala_especial",
sala_default}]):
    disciplinas[_d[0]] = {
```

```

        "carga" : _d[2],
        "duplo": 1 if _d[3] == "sim" else 0,
        "professor": _d[1],
        "sala": _d[4]}

    if _d[1] not in professores:
        professores[_d[1]] = {"disciplinas": [], "disponibilidade":{}}
    professores[_d[1]]["disciplinas"].append(_d[0])

for disp in read_data("dados/disponibilidade_excecoes.csv"):
    if disp[1] not in professores[disp[0]]["disponibilidade"]:
        professores[disp[0]]["disponibilidade"][disp[1]] = []
    ##range(periodos) 0..4 e os periodos do csv começa em 1
    ## por isso o disp[2] - 1
    professores[disp[0]]["disponibilidade"][disp[1]].append(disp[2] - 1)

```

```

In [ ]: horario = pywraplp.Solver.CreateSolver('SCIP')

## x[t][d][p][dis] = 1 se a turma t tem a disciplina dis no dia d, no tempo
p
x = {}
for _t in turmas:
    x[_t] = {}
    for _d in dias:
        x[_t][_d] = {}
        for _p in range(periodos):
            x[_t][_d][_p] = {}
            for _dis in disciplinas:
                x[_t][_d][_p][_dis] = horario.IntVar(0, 1,
f"x_{_t}_{_d}_{_p}_{_dis}")

def get_Value(t,d,p,dis):
    return x[t][d][p][dis]

```

Uma turma não pode ter duas aulas em simultâneo. (R1)

$$R1: \forall t, d, p : \sum_{c \in C} x_{t,d,p,c} \leq 1$$

```

In [ ]: for _t in turmas:
        for _d in dias:
            for _p in range(periodos):
                horario.Add(sum(get_Value(_t,_d,_p,_dis) for _dis in
disciplinas) <= 1)

```

Cada disciplina cumpre exatamente a carga semanal, para cada turma. (R2)

$$R2: \forall t, c : \sum_{d \in D} \sum_{p \in P} x_{t,d,p,c} = \text{carga}_c$$

```

In [ ]: for _t in turmas:
        for _dis in disciplinas:

```

```
horario.Add(sum(get_Value(_t, _d, _p, _dis) for _d in dias for _p
in range(periodos)) == disciplinas[_dis]["carga"])
```

Para as disciplinas sem duplo período, no máximo um tempo por dia em cada turma. (R3)

$$R3: \forall t, d, \forall c \notin C_2: \sum_{p \in P} x_{t,d,p,c} \leq 1$$

```
In [ ]: for _t in turmas:
        for _d in dias:
            for _dis in disciplinas:
                if not disciplinas[_dis]["duplo"]:
                    horario.Add(sum(get_Value(_t, _d, _p, _dis) for _p in
range(periodos)) <= 1)
```

Disciplinas marcadas `duplo_periodo=sim` só podem ser dadas em blocos de 2 tempos consecutivos, no mesmo dia. (R4)

$$R4: \forall c \in C_2, t, d: \sum_{p \in P} x_{t,d,p,c} \leq 2 x_{t,d,p,c} \leq x_{t,d,p-1,c} + x_{t,d,p+1,c}$$

```
In [ ]: for _dis in disciplinas:
        if disciplinas[_dis]["duplo"]:
            for _t in turmas:
                for _d in dias:
                    ocorre = [get_Value(_t, _d, _p, _dis) for _p in
range(periodos)]
                    horario.Add(sum(ocorre) <= 2)
                    for _p in range(periodos):
                        if _p == 0:
                            horario.Add(get_Value(_t, _d, _p, _dis) <=
get_Value(_t, _d, _p+1, _dis))
                        elif _p == periodos-1:
                            horario.Add(get_Value(_t, _d, _p, _dis) <=
get_Value(_t, _d, _p-1, _dis))
                        else:
                            horario.Add(get_Value(_t, _d, _p, _dis) <=
get_Value(_t, _d, _p-1, _dis) + get_Value(_t, _d, _p+1, _dis));
```

Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas ou disciplinas diferentes. (R5)

$$R5: \forall q, d, p: \sum_{t \in T} \sum_{c \in C_q} x_{t,d,p,c} \leq 1$$

```
In [ ]: for _prof in professores:
        for _d in dias:
            for _p in range(periodos):
```

```

horario.Add(sum(get_Value(_t,_d,_p,_dis) for _t in turmas
for _dis in professores[_prof]["disciplinas"] ) <= 1)

```

Um professor só pode dar aulas nos tempos em que está disponível. (R6)

$$R6: \forall q, \forall (d, p) \in I_q : \sum_{t \in T} \sum_{c \in C_q} x_{t,d,p,c} = 0$$

```

In [ ]: for _prof in professores:
        for _d in professores[_prof]["disponibilidade"]:
            for _p in professores[_prof]["disponibilidade"][_d]:
                horario.Add(sum(get_Value(_t,_d,_p,_dis) for _t in turmas
for _dis in professores[_prof]["disciplinas"]) == 0)

```

Cada aula ocupa uma sala. Disciplinas com `sala_especial` só podem usar salas desse tipo; as restantes usam salas `normal`. Em nenhum tempo o número de aulas a decorrer num tipo de sala pode exceder a `quantidade` desse tipo definida em `salas.csv`. (R7)

$$R7: \forall s, d, p : \sum_{t \in T} \sum_{c \in C_s} x_{t,d,p,c} \leq Q_s$$

```

In [ ]: for sala in salas:
        disciplinas_sala = [_dis for _dis in disciplinas if disciplinas[_dis]
["sala"] == sala]
        for _d in dias:
            for _p in range(periodos):
                horario.Add(sum(get_Value(_t,_d,_p,_dis) for _t in turmas
for _dis in disciplinas_sala) <= salas[sala])

```

Resolução do modelo

Depois de adicionadas as restrições R1 a R7, o modelo é resolvido com `horario.Solve()`, usando o solver SCIP (programação linear inteira).

```

In [ ]: stat = horario.Solve()

if stat == pywraplp.Solver.OPTIMAL:
    print("Solução encontrada!")
else:
    print("Sem Solução")

```

Solução encontrada!

Representar o horário gerado de acordo com as restrições

```
In [ ]: horas = [f"{8 + p}:00" for p in range(periodos)]

def apresentar_horario(turmas):
    for turma in turmas:
        print(f"\n\tTurma {turma}")
        print(f"{'':8}" + "".join(f"{d:^12}" for d in dias))
        for p in range(periodos):
            linha = f"{horas[p]:8}"
            for d in dias:
                coluna = "-"
                for dis in disciplinas:
                    if int(get_Value(turma, d, p, dis).solution_value()):
                        coluna = dis
                linha += f"{coluna:^12}"
            print(linha)

apresentar_horario(turmas)
```

	Turma 7ºA				
	Seg	Ter	Qua	Qui	Sex
8:00	Inglês	Inglês	Ciências	Ciências	-
9:00	Ciências	-	-	-	-
10:00	-	-	-	História	História
11:00	Educação Física	Matemática	Português	Português	Matemática
12:00	Educação Física	Português	Matemática	Matemática	Português

	Turma 7ºB				
	Seg	Ter	Qua	Qui	Sex
8:00	Ciências	-	-	Inglês	-
9:00	Inglês	Ciências	História	-	-
10:00	-	História	Ciências	-	-
11:00	Matemática	Educação Física	Matemática	Matemática	Português
12:00	Português	Educação Física	Português	Português	Matemática

Ferramentas de Apoio

- Material das aulas práticas (ficha 3)
- [Pandas \(csv\)](#)

Ferramentas de LLM

- [Chat Claude](#)